Object Oriented Design Patterns II (JIAT/OODP II / EX / 01)

වීඩියෝව **Record** කිරීම සඳහා මූලික උපදෙස් (**General Guidelines & Setup**)

- **කාල සීමාව (Duration):** විනාඩි 10 ත් 15 ත් අතර (Strictly 10–15 Minutes). මීට වඩා අඩු හෝ වැඩි නොවිය යුතුය.
- **වීඩියෝ Format එක:** MP4 (Resolution: 1080p Full HD හෝ 720p).
- **Recording Software:** OBS Studio (Recommended) හෝ Windows Game Bar (`Win + G`) භාවිතා කළ හැක.
- **Audio / කටහඬ:** Background noise අවම කර, පැහැදිලි Headset එකක් හෝ Mic එකක් භාවිතා කරන්න.
- **භාෂාව (Language):** Presentation එක සම්පූර්ණයෙන්ම **English** වලින් සිදු කළ යුතුය. ඔබ කතා කළ යුතු සම්පූර්ණ Script එක පහත කොළ පැහැති කොටු (**Speech Box**) වල ලබා දී ඇති බැවින් එය බලාගෙන පැහැදිලිව කියවන්න.
- **Screen Activity:** කතා කරන අතරතුර තිරයේ කළ යුතු දේවල් කහ පැහැති කොටු (**Activity Box**) වල Sinhala සහ English වලින් පැහැදිලි කර ඇත.
- **Application එක Run කරන ආකාරය:** Terminal / CMD එක open කර `cd D:\SemiOODPII\SmartCity360` ගොස් `mvn compile exec:java` run කර Menu එක ලබා ගන්න.

Scene 1: Introduction & System Architecture Overview 00:00 - 01:30 (1.5 Mins)

තිරයේ කළ යුතු දේ (On-Screen Activity):
• IDE (IntelliJ IDEA, VS Code හෝ Eclipse) එක Screen එකේ පෙන්වන්න.
• වම් පැත්තේ ඇති Project Explorer එකෙන් Package structure එක (`com.smartcity360.security`, `zones`, `complaints`, `streetlights`, `policy`, `app`) expand කර පෙන්වන්න.
• Terminal එකේ `SmartCity360App` run කර Welcome Banner එක පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (Spoken Narration Script): "Good day, Respected Evaluators. My name is [Your Name], and today I am demonstrating the SmartCity360 Municipal Management Platform, developed for the Object Oriented Design Patterns II Final Assessment.

As a Senior Software Architect at UrbanTech Solutions, my objective was to design a decoupled, maintainable, and memory-efficient platform for smart municipal management. As you can see in the project tree, the solution systematically implements five Gang of Four (GoF) design patterns: Decorator, Composite, Chain of Responsibility, Flyweight, and Interpreter.

Let us now examine each design pattern in action through our live interactive demonstration system."

Pro-Tip: කතා කරද්දී Cursor එකෙන් අදාළ packages උඩට mouse එක ගෙනියන්න.

Scene 2: Part A – Dynamic Security Wrappers for Data Streams (Decorator) 01:30 - 04:00 (2.5 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity):**
• IDE එකේ `DataStream.java`, `MeterDataStream.java`, සහ `DataStreamDecorator.java` files පෙන්වන්න.
• `EncryptionDecorator.java`, `CompressionDecorator.java`, `AuthenticationDecorator.java` classes කෙටියෙන් පෙන්වන්න.
• Interactive Menu එකේ **Option 1** type කර [ENTER] ඔබන්න.
• Terminal එකේ output එකෙන් *Encrypted*, *Compressed*, *HMAC Attached*, සහ අවසානයේ *Verification Result: true* වන ආකාරය Cursor එකෙන් highlight කර පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script):** "We begin with Part A: Dynamic Security Wrappers for Data Streams, where we applied the Decorator Pattern.

Energy meters transmit critical power usage telemetry that must be secured with AES-128 Encryption, GZIP Compression, and HMAC Authentication. If we used static inheritance, supporting these optional features would result in an explosion of eight subclasses.

By using the Decorator Pattern, our base component 'MeterDataStream' remains completely untouched, satisfying the Open/Closed Principle. In our runner code, we dynamically wrap the stream with Authentication, Compression, and Encryption.

Now, observing the terminal output for Option 1: During transmission, the payload is authenticated with a SHA-256 signature, compressed to save bandwidth, and encrypted with AES-128. During reception, the layers are unwrapped in reverse order: decrypted, decompressed, and validated against tampering. As shown here, data integrity is 100% verified."

Pro-Tip: "Russian Matryoshka doll" උපමාව මතක් කරන්න. Core stream එක වෙනස් නොවී dynamic layers wrap වන බව එmphාස් කරන්න.

Scene 3: Part B – Hierarchical Structure of Smart Zones (Composite)
04:00 - 06:30 (2.5 Mins)

තිරයේ කළ යුතු දේ **(On-Screen Activity):**
• IDE එකේ `ZoneComponent.java`, `Building.java` (Leaf), සහ `ZoneComposite.java` (Composite) open කර පෙන්වන්න.
• Interactive Menu එකේ **Option 2** type කර [ENTER] ඔබන්න.
• Terminal එකේ print වන Structural Hierarchy Tree එක (Greater Colombo -> Colombo 01 & 05 -> Streets -> Buildings) පෙන්වන්න.
• Colombo 01 වලට 'shutdownPower()' command එක යැවූ පසු එහි usage 0 වී, මුළු city usage එක 2200 kWh සිට 340 kWh දක්වා අඩුවන ආකාරය පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් **(Spoken Narration Script):** "Moving to Part B: Hierarchical Structure of Smart Zones, where we implemented the Composite Pattern.

A smart city is arranged in a natural spatial tree: City contains Districts, Districts contain Streets, and Streets contain Buildings. We need commands like 'shutdownPower()' and 'reportUsage()' to execute uniformly whether invoked on a single building or an entire metropolitan district.

Our 'ZoneComponent' interface defines these operations. 'Building' is a Leaf node with no children, while 'ZoneComposite' holds a collection of ZoneComponents and recursively delegates commands.

In the terminal output for Option 2: Notice the visual hierarchy tree. The initial city power draw is 2,200 kilowatt-hours. When we execute 'shutdownPower()' specifically on the Colombo 01 Business District, the command automatically cascades down through York Street and Galle Road, deactivating all four buildings. The city power draw drops immediately to 340 kWh, while Colombo 05 remains fully energized. When power is restored, the usage returns to 2,200 kWh. This demonstrates seamless recursive delegation without client-side type checking."

Pro-Tip: Client එකට `instanceof` හෝ `if(isDistrict)` check කිරීමට අවශ්‍ය නොවන බව (Uniform treatment) පවසන්න.

Scene 4: Part C – Escalation of Citizen Complaints (Chain of Responsibility) 06:30 - 09:00 (2.5 Mins)

තිරයේ කළ යුතු දේ **(On-Screen Activity):**
• IDE එකේ `ComplaintHandler.java` base class එක සහ `LocalStaffHandler`, `SupervisorHandler`, `CityManagerHandler` පෙන්වන්න.
• Interactive Menu එකේ **Option 3** type කර [ENTER] ඔබන්න.
• Complaints 3 ක් process වන ආකාරය පෙන්වන්න:
1. Trash issue (LOW) → Local Staff resolve කරයි.
2. Water pipe leak (MEDIUM) → Local staff skip කර Supervisor resolve කරයි.

3. Transformer explosion (CRITICAL) → Local Staff → Supervisor → City Manager වෙත escalate වී resolve වන ආකාරය පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (Spoken Narration Script): "Next is Part C: Citizen Complaint Escalation, modeled using the Chain of Responsibility Pattern.

Citizen complaints range from routine maintenance to city emergencies. Directly coupling the citizen portal to specific administrative officers creates rigid and fragile code. By using the Chain of Responsibility, we decouple the sender of the complaint from the receiver by passing requests along a chain of handlers.

Looking at our console output for Option 3: First, complaint 101 regarding an overflowing trash bin has LOW severity. The 'Local Staff Field Unit' accepts and resolves it immediately.

Second, complaint 102 involves a burst water pipe with MEDIUM severity. The Local Staff determines it exceeds their authority and passes it to the 'District Operations Supervisor', who authorizes the repair.

Third, complaint 103 is a CRITICAL electrical explosion. Notice the automatic chain traversal: Local Staff passes to Supervisor, and Supervisor escalates to the 'City General Manager', who triggers the emergency protocol. The final audit summary verifies that every complaint has a recorded handler and resolution action."

Pro-Tip: Traceability සඳහා Complaint object එක තුළම Audit Trail එකක් maintain වන බව පෙන්වන්න.

Scene 5: Part D – Smart Streetlights Management & Benchmark (Flyweight) 09:00 - 11:30 (2.5 Mins)

තිරයේ කළ යුතු දේ (On-Screen Activity):
• IDE එකේ `StreetLightType.java` (Intrinsic State) සහ `StreetLight.java` (Extrinsic State) පෙන්වන්න.
• `StreetLightFactory.java` හි cache map එක පෙන්වන්න.
• Interactive Menu එකේ **Option 4** type කර [ENTER] ඔබන්න.
• Terminal එකේ **50,000 Streetlights Memory Benchmark** එක highlight කර පෙන්වන්න (108 MB vs 4.74 MB, ~95.6% memory savings).

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (Spoken Narration Script): "Now let us examine Part D: Efficient Management of Smart Streetlights, using the Flyweight Pattern.

A modern city operates tens of thousands of streetlights. Every single light shares identical hardware specifications: bulb model, wattage, color temperature, and calibration curves. However, their GPS coordinates, power status, and dimming percentages are unique.

We partitioned the state into Intrinsic and Extrinsic. The Intrinsic State—which includes heavy 2KB calibration data—is encapsulated in the immutable 'StreetLightType' class and cached by

'StreetLightFactory'. The Extrinsic State—such as coordinates and power—remains in the lightweight 'StreetLight' context object.

In the terminal benchmark for Option 4: We scaled the simulation to 50,000 physical fixtures. Without the Flyweight pattern, duplicating hardware specifications consumes 108.01 megabytes of JVM heap memory. With the Flyweight pattern, all 50,000 fixtures point to a single cached flyweight instance, reducing memory consumption to just 4.74 megabytes.

This achieves a verified 95.6% memory reduction, preventing OutOfMemory errors and garbage collection latency at city-wide scale."

Pro-Tip: මෙම 95.6% memory reduction එක Examiners ලා ඉතාම ප්‍රිය කරන practical proof එකකි. එය පැහැදිලිව පෙන්වන්න.

Scene 6: Part E – City Policies Defined in DSL (Interpreter Pattern) 11:30 - 13:30 (2.0 Mins)

තිරයේ කළ යුතු දේ **(On-Screen Activity):**
• IDE එකේ `Context.java`, `Expression.java`, `NumericComparisonExpression.java`, සහ `PolicyParser.java` පෙන්වන්න.
• Interactive Menu එකේ **Option 5** type කර [ENTER] ඔබන්න.
• DSL Rules (උදා: `noise_level > 70 AND hour >= 22`) Abstract Syntax Tree එකකට convert වී dynamic sensor context evaluate වන ආකාරය පෙන්වන්න.

මයික්‍රෆෝනයෙන් කිව යුතු දෙබස් (Spoken Narration Script): "For Part E: Interpreting City Policies Defined in DSL, we implemented the Interpreter Pattern.

Municipal regulations, such as night-time noise curfews and commercial parking limits, change frequently. Hardcoding these rules as nested Java if-else statements requires rebuilding and redeploying the server for every policy change.

We designed a Domain-Specific Language (DSL) mini-language. Our 'PolicyParser' translates plain-text rules into an Abstract Syntax Tree of 'Expression' objects. Terminal expressions evaluate relational conditions like greater-than or equals, while Non-Terminal expressions evaluate logical ANDs and ORs against dynamic sensor telemetry in the 'Context'.

Observing terminal Option 5: Rule 1 specifies 'noise_level > 70 AND hour >= 22'. When tested against late-night residential telemetry at 11 PM with 82.5 decibels, it interprets as TRUE, automatically triggering a police dispatch alert. Against daytime telemetry, it evaluates as FALSE. Furthermore, overtime parking rules in commercial zones trigger automated parking citation tickets. This allows city administrators to update operational rules dynamically without touching a single line of backend code."

Pro-Tip: Text rule එකක් Runtime එකේදී Code compile නොකර evaluate කළ හැකි බව අවධාරණය කරන්න.

Scene 7: Automated JUnit Test Suite, Build Verification & Conclusion

13:30 - 15:00 (1.5 Mins)

තිරයේ කළ යුතු දේ (**On-Screen Activity):**
• Terminal එකේ `mvn clean test` run කරන්න.
• Automated Tests 6 ම pass වී **BUILD SUCCESS** ලැබෙන අයුරු පෙන්වන්න.
• Interactive App එකේ **Option 6 (Full End-to-End Simulation)** කෙටියෙන් run කර පෙන්වන්න.
• අවසානයේ Option 7 මඟින් Exit වී ස්තූතිවන්ත වන්න.

මයික්‍රොෆෝනයෙන් කිව යුතු දෙබස් (**Spoken Narration Script):** "To guarantee system stability, we engineered a comprehensive automated test suite. Let us execute 'mvn clean test'.

As you can observe in the build output: All six unit tests pass with zero failures and zero errors in less than a second, concluding with Maven BUILD SUCCESS. The tests rigorously validate tamper detection in the Decorator, power cascading in the Composite, multi-tier escalation in the Chain of Responsibility, object identity caching in the Flyweight, and complex Boolean logic in the Interpreter.

In conclusion, SmartCity360 successfully applies the five Gang of Four design patterns to resolve critical architectural challenges in modularity, extensibility, loose coupling, memory optimization, and dynamic rule interpretation.

Thank you very much for your time and evaluation."

Pro-Tip: "Thank you very much for your time and evaluation" යැයි පවසා recording එක stop කරන්න.

Video Record කළ පසු කළ යුතු දේ (**After Recording):**

1. Video file එක `.mp4` format එකෙන් save වී ඇත්දැයි බලන්න.
2. Video එක play කර Audio එක පැහැදිලිව ඇසෙනවාද, Code සහ Terminal screen එක පැහැදිලිව පෙනෙනවාදැයි පරීක්ෂා කරන්න.
3. කාලය විනාඩි 10 ත් 15 ත් අතර පවතී දැයි තහවුරු කරගන්න.
4. Student Portal එකේ **Demonstration Video** section එකට upload කරන්න.